LearnLens — System Architecture

How Frontend, Backend, AI Integrations, and Infrastructure connect at runtime

🖥️ **FRONTEND** | ⚙️ **BACKEND** | 🤖 **AI LAYER** | 🛠️ **INFRA**

END-TO-END REQUEST FLOW

⌨️ **User submits topic** ▶

⚙️ **Backend creates job**
Returns job_id instantly ▶

🗄️ **Enqueue to Redis**
Redis queue / infra layer ▶

🧠 **Worker dequeues**
Gemini expands prompt ▶

🤖 **AI generates media**
Lyria / Imagen / Veo ▶

☁️ **R2 stores file**
Returns CDN URL ▶

📡 **WebSocket fires**
CDN URL to browser ▶

🖼️ **Render result**
Frontend displays

PERSON 1 · NEXT.JS

🏠 Landing Page & Input UI

▶ Hero + CTA

🎛️ Topic + Mode picker

▶ POST /generate

📨 Job ID received

▶ instant response → open WebSocket

⏳ Progress Bar

▶ live WS events

🖼️ Result Viewer

▶ "done" event → image / audio / video

PERSON 2 · FASTAPI

📩 POST /generate

▶ creates job ID

🗄️ Redis Queue

▶ enqueue job payload

⚙️ Worker Process

▶ worker picks up (always running)

🧠 Gemini Route

▶ API call: expand topic → prompt

📝 Route to AI

▶ structured prompt by output type

PERSON 3 · LYRIA · IMAGEN · VEO

🖼️ Imagen 3

image bytes

🎵 Google Lyria

audio bytes + lyrics

🎬 Veo 2 / Runway

video bytes

▶ bytes returned → upload_media()

☁️ upload_media()

▶ CDN URL back via Redis pub/sub

📡 Emit "done"

PERSON 4 · REDIS · R2 · CDN · WS

🗄️ Redis Queue

job store

▶

🔌 WebSocket Server

/ws/[job_id]

▶

📡 Redis Pub/Sub

pub/sub progress events

▶

☁️ Cloudflare R2

image / audio / video

▶

🌐 CDN

global media URLs

Caching shortcut: Before calling any AI API, the worker checks Redis for a cache key `cache:{type}:{topic}`. On a hit, it skips Gemini and all AI calls entirely, fires *done* immediately with the cached CDN URL — makes stage presentations instant.

Share links: `/s/[job_id]` on the frontend calls `GET /output/[job_id]` on the backend, which reads the CDN URL from Redis and returns it — no re-generation needed.